COMP3011: Technical Report

Project Title: Transport Delay Analytics API

Author: Araav Bhardwaj

Module: Web Services and Web Data (COMP3011)

1. Project Links & Deliverables

- **GitHub Repository:**

  https://github.com/araavbhardwaj/transport-delay-api?tab=readme-ov-file

- **API Documentation (PDF):**

  https://github.com/araavbhardwaj/transport-delay-api/blob/main/api_documentation.pdf

- **Presentation Slides:**

  https://docs.google.com/presentation/d/1LHvpjMMOdgxHQpxmb3xYdDMSosEc7-DNc1jHI23-80M/edit?usp=sharing

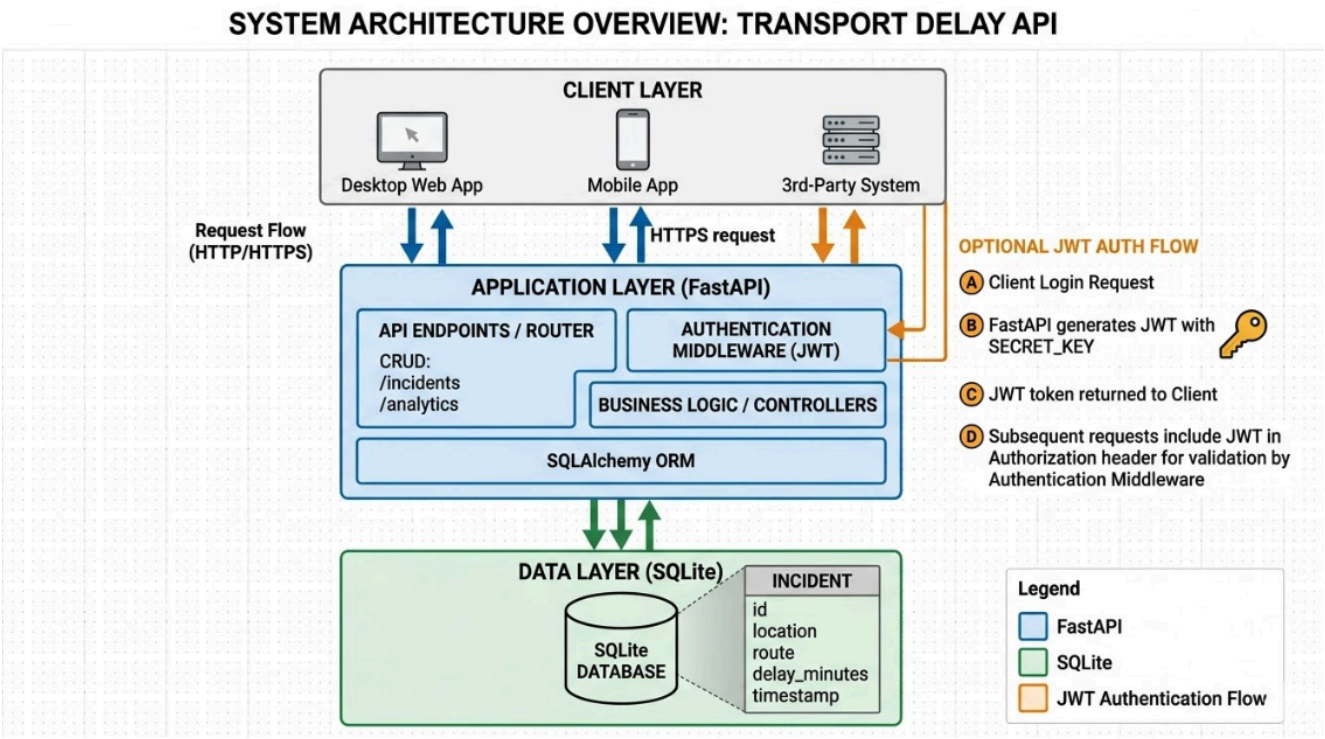
2. Introduction & Project Overview

This project presents a fully functional, data-driven RESTful API designed to manage and analyze public transport delay data. While fulfilling the required CRUD functionality, the system extends beyond basic data storage by providing analytical endpoints that identify delay patterns, temporal trends, and transport reliability insights. The API is designed using modern software engineering principles, including modular architecture, secure authentication, database-level computation, and automated testing. Overall, the system demonstrates both practical API development and higher-level analytical capabilities.

3. System Architecture

The system follows a layered architecture consisting of three primary components: the client layer, the application layer, and the data layer. The client layer sends HTTP requests via tools such as Postman, Swagger UI, or a web browser. The application layer, powered by FastAPI, handles the routing, input validation, authentication, and core business logic. Finally, the data layer utilizes SQLite via SQLAlchemy to safely store and process the transport delay data.

During a standard request flow, FastAPI validates the incoming HTTP request using Pydantic schemas, verifies the JWT token for protected routes, interacts with the SQLite database via the ORM, and returns a structured JSON response. This architecture ensures a strict separation of concerns, improving scalability and maintainability.



4. API Design & Functionality

The API follows RESTful principles and provides both core CRUD operations and advanced analytical endpoints. The primary data management endpoints allow users to create a new record (`POST /incidents/`), retrieve all records (`GET /incidents/`), update an existing entry (`PUT /incidents/{id}`), and remove a record (`DELETE /incidents/{id}`). To extract meaningful insights, analytical endpoints such as `/analytics/hotspots` and `/analytics/trends/recent` were implemented to identify the routes with the highest delays and analyze recent temporal patterns. The entire API relies on structured JSON request and response formats, strictly enforces input validation, returns standard HTTP status codes (200, 401, 404), and secures protected endpoints using JWT authentication.

Example Request & Response:


```
[
  {
    "location": "Leeds City Centre",
    "total_incidents": 56,
    "total_delay_minutes": 1998,
    "average_delay": 35.68
  },
  {
    "location": "Burley",
    "total_incidents": 53,
    "total_delay_minutes": 1653,
    "average_delay": 31.19
  }
]
```


5. Data Model Design

The system utilizes a relational database schema designed for efficient querying and downstream analytics. The primary table, `Incident`, is anchored by an integer `id` serving as the Primary Key. The model captures the context of the delay using string fields for the `location`, `route`, and a specific `description` of the event. Furthermore, it records the exact `delay_minutes` as an integer and the `timestamp` as a datetime object. This simple yet structured schema ensures highly efficient CRUD operations while supporting the complex aggregation queries required by the analytics engine.

6. Technology Stack & Justification

FastAPI was selected over Django as the primary framework due to its exceptional performance, asynchronous capabilities, and minimalistic footprint. Crucially, FastAPI provides

built-in OpenAPI documentation generation (Swagger and ReDoc) and deep integration with Pydantic, which significantly streamlined input validation. SQLite was chosen as the database engine because it is lightweight, highly portable, and requires minimal setup overhead, making it ideal for a prototype-scale application. To interact with SQLite, SQLAlchemy was implemented as the Object-Relational Mapper (ORM). Using an ORM abstracts the database logic, improves maintainability, and actively prevents SQL injection vulnerabilities compared to writing raw SQL strings. Finally, JWT (JSON Web Token) authentication, implemented via OAuth2PasswordBearer, was chosen to provide a stateless, secure authentication mechanism, with all cryptographic keys safely stored in hidden environment variables.

7. Advanced Features & Performance Considerations

To elevate the API beyond basic CRUD functionality, several advanced features were implemented to optimize performance. Most notably, database-level analytics were engineered using SQLAlchemy functions (such as `func.sum`and `func.avg`). By calculating totals and averages directly within the database query, the system avoids loading large, memory-intensive datasets into the Python application layer, vastly improving execution speed and scalability. Additionally, to demonstrate these analytics in a meaningful way, a custom data seeding script was developed. This script populates the database with 250 realistic, historically distributed records, allowing for robust testing of the analytical endpoints under simulated real-world conditions.

8. Testing & Error Handling

A comprehensive automated testing strategy was implemented using `pytest` and `httpx`. The testing suite provides broad coverage, explicitly verifying endpoint accessibility, validating the structure of JSON responses, checking authentication barriers, and ensuring the mathematical correctness of the analytical endpoints. Furthermore, strict error handling is enforced across the application. The API returns structured `401 Unauthorized` responses for invalid or missing tokens and `404 Not Found` responses when querying nonexistent resources, ensuring the server handles bad requests gracefully without crashing.

9. Trade-offs & Design Decisions

Several key architectural trade-offs were evaluated during development. FastAPI was chosen over Django to prioritize performance and development speed; while Django offers a more extensive suite of built-in features, its monolithic nature was deemed too heavy for a focused, data-driven API. For data storage, SQLite was favored for its simplicity and portability during development, though PostgreSQL would be the preferred choice for a large-scale, concurrent production deployment. Finally, utilizing SQLAlchemy instead of raw SQL queries sacrificed some fine-grained query control, but this trade-off was gladly accepted to gain significantly improved code safety, maintainability, and built-in protection against injection attacks.

10. Limitations & Future Development

The primary limitation of the current system is its reliance on a static, synthetic dataset; the API does not currently ingest real-time transport data. Additionally, while SQLite is perfect for the current scope, it would become a bottleneck under high concurrent read/write loads in a production environment. Future improvements would include migrating the database to PostgreSQL and integrating a scheduled background task to fetch live data from public APIs, such as the UK Department for Transport datasets. Expanding the system to cross-reference delays with live weather data via the OpenWeatherMap API would also provide much deeper analytical insights.

11. Generative AI Declaration & Analysis

Declaration: Generative AI (Google Gemini) was used integrally throughout this project as an architectural support and development tool.

Analysis of Usage:

Generative AI tools, including ChatGPT and Google Gemini, were used extensively throughout the development lifecycle in a structured and transparent manner. Rather than being limited to code generation, AI was primarily employed to explore high-level architectural decisions, including comparisons between REST and alternative API paradigms, and the evaluation of frameworks such as FastAPI and Django based on performance, scalability, and documentation capabilities. It also supported the design of an efficient relational data model and the generation of realistic synthetic datasets to enable meaningful analytics. A key contribution of AI was in reimagining the implementation of analytical features, transitioning from inefficient Python-based processing to optimised SQL-level aggregations using SQLAlchemy, significantly improving performance and scalability. Additionally, AI assisted in debugging, refactoring, and improving code structure, as well as enhancing the clarity and professionalism of documentation. All AI-generated outputs were critically evaluated, adapted, and validated to ensure correctness and alignment with project requirements, demonstrating a creative and high-level application of Generative AI in line with modern software engineering practices.

12. References

- FastAPI Documentation: https://fastapi.tiangolo.com/
- SQLAlchemy Documentation: https://www.sqlalchemy.org/
- OpenAPI Specification: https://swagger.io/specification/
- Pytest Documentation: https://docs.pytest.org/